

Human Vision – Topic 5

White Balance and Color Constancy

Yonghao Lee

May 16, 2026

Contents

1	Recap: From Retinex to White Balance	2
1.1	The Lambertian Model, One More Time	2
1.2	The Goal, Precisely	2
2	The Fundamental Ambiguity	3
2.1	Counting Unknowns	3
3	Algorithms for White Balance	4
3.1	The Gray-World Algorithm	4
3.2	The Brightest-Pixel-Is-White Algorithm	5
3.3	Comparison on a Synthetic Example	5
4	What Do Humans Use?	6
5	Color Constancy: The RGB Extension	6
5.1	The Simplified Color Model	6
5.2	The Color Constancy Problem	7
5.3	Algorithms: Just Run the Scalar Version Three Times	7
5.4	A Worked Example That Goes Wrong	8
5.5	The Famous Example: #thedress	9
5.6	Less Famous but Equally Telling: The Yellowing Demonstration	9
6	Putting It All Together	10
6.1	Comparing Retinex and White Balance Side by Side	10
6.2	Beyond the Two Textbook Algorithms	10
6.3	Summary	10

1 Recap: From Retinex to White Balance

In the previous topic we met the *intrinsic image* programme of Barrow and Tenenbaum: the idea that the visual system computes a midlevel decomposition of an image $I(x, y)$ into a reflectance map $R(x, y)$ and an illumination map $L(x, y)$, related multiplicatively by

$$I(x, y) = R(x, y) \cdot L(x, y).$$

The Retinex algorithm offered one approach to this factorization. It exploited the statistical observation that, in natural scenes, reflectance edges tend to be *sharp* and illumination changes tend to be *soft*. Thresholding the gradient of the log-image $i = l + r$ separates the two contributions and then integrating each piece back recovers the maps.

In this topic we look at a closely related but distinctly different version of the same inverse problem. Instead of trying to recover the full illumination *map* $L(x, y)$ – which varies from pixel to pixel due to shading and shadows – we restrict attention to scenes where the illumination is a single global quantity L . We assume the whole scene is lit by one common light source, and ask: what was the color of the light, and what is the underlying paint color of each surface?

This is the **white balance** problem. Its three-channel extension to RGB images is called **color constancy**, and it is one of the cleanest examples of an underdetermined inverse problem in vision.

Why You Should Care

Every camera you have ever used solves a version of this problem. Photograph a piece of white paper indoors under a warm tungsten bulb and the raw sensor reading is reddish; photograph the same paper outdoors under an overcast sky and the reading is bluish. Yet you, looking at the paper directly, see it as *white* in both cases. Your brain – like the camera’s auto-white-balance algorithm – has to estimate the color of the light and “divide it out” to recover the underlying surface color. This topic is about how both the brain and the camera might do that.

1.1 The Lambertian Model, One More Time

For a Lambertian surface lit by a single light source,

$$I = \rho L \cos(\theta),$$

where I is the intensity reaching the eye (or camera), $\rho \in [0, 1]$ is the surface’s **albedo**, L is the intensity of the incident light, and θ is the angle between the surface normal and the light direction. For this topic we assume the geometry is fixed across the patch we are looking at, so the $\cos(\theta)$ factor is absorbed into L and the model simplifies to

$$I = \rho L.$$

We will treat ρ and L as scalars for most of the chapter and let them become RGB triples in Section 5.

1.2 The Goal, Precisely

Definition 1.1 The White Balance Problem

Given an image $I(x, y)$ produced by the model

$$I(x, y) = \rho(x, y) \cdot L,$$

recover the reflectance map $\rho(x, y)$ and the single scalar illuminant L .

This is a stripped-down version of the intrinsic image problem from last lecture. The illumination has been collapsed from a spatially varying map $L(x, y)$ to a single number L . That simplification makes the problem more tractable, but it does not make it well-posed.

2 The Fundamental Ambiguity

2.1 Counting Unknowns

Suppose the image has N pixels. The model

$$I(x, y) = \rho(x, y) \cdot L$$

has $N + 1$ **unknowns**: one albedo at each of N pixels, plus one global illuminant. We have N observed pixel values. We are exactly one constraint short of a square system, and the gap cannot be closed without an extra assumption.

This is structurally the same “two numbers multiplying to 64” game from last lecture, only now the entire scene shares a single mystery multiplier rather than one multiplier per pixel.

A Worked Example of the Ambiguity

Consider a tiny one-dimensional image:

$$I = (100, 125, 125, 150).$$

This image admits (at least) two perfectly reasonable interpretations.

Interpretation A – bright illumination, dark surfaces. Suppose $L = 255$ (full sunlight). Dividing through gives

$$\rho = \left(\frac{100}{255}, \frac{125}{255}, \frac{125}{255}, \frac{150}{255} \right) \approx (0.39, 0.49, 0.49, 0.59).$$

Under this reading, the middle pixel ($I = 125$) is a roughly mid-gray surface.

Interpretation B – dim illumination, bright surfaces. Suppose instead $L = 150$ (an indoor lamp). Then

$$\rho = \left(\frac{100}{150}, \frac{125}{150}, \frac{125}{150}, \frac{150}{150} \right) \approx (0.67, 0.83, 0.83, 1.00).$$

Now the middle pixel is an off-white surface and the rightmost is a perfect white.

The image is identical – the same four numbers. The interpretation depends entirely on what we choose to believe about L . The middle pixel (125) is “gray” under interpretation A and “white-ish” under interpretation B, and the pixel values alone cannot decide between them.

This is exactly why we need an algorithm. Pixel values cannot tell us the answer. We must bring in a prior assumption about what scenes typically look like, and let that assumption pin down L .

3 Algorithms for White Balance

The general shape of every white-balance algorithm is the same:

1. Use some heuristic about “what scenes typically look like” to estimate L .
2. Once L is known, recover $\rho(x, y) = I(x, y)/L$ trivially.

The art lies entirely in step 1. We look at the two simplest and most influential choices.

3.1 The Gray-World Algorithm

The Gray-World Assumption

On average, the world is gray. The mean reflectance across a typical scene is 0.5.

If we believe this, taking the average of both sides of $I = \rho L$ gives

$$\frac{1}{N} \sum_{x,y} I(x, y) = L \cdot \frac{1}{N} \sum_{x,y} \rho(x, y) = L \cdot 0.5,$$

which we solve for L :

$$L = \frac{2}{N} \sum_{x,y} I(x, y).$$

The reflectance map is then recovered pixel-by-pixel as $\rho(x, y) = I(x, y)/L$.

Why Gray-World Works (Sometimes)

The assumption is empirically defensible. Photographs averaged over many subjects do tend to converge to a mid-gray average. Point a camera at a busy outdoor scene – trees, sky, pavement, people, cars – and the mean pixel is rarely far from 0.5.

But the assumption is fragile. Point the same camera at a red rose against a black background and the mean is dominated by red. Gray-world will then conclude that the illumination must have been red and dutifully “correct” the rose into a gray-ish thing. The algorithm has no way of knowing it has been tricked, because the only thing it ever sees is the image.

In practice this is why automatic white balance on cameras can fail dramatically on heavily monochromatic scenes – a sunset, a forest fire, a snow field. The averaging prior simply does not hold.

3.2 The Brightest-Pixel-Is-White Algorithm

The Brightest=White Assumption

The brightest patch in the scene corresponds to a perfectly white surface, i.e., $\rho = 1$ somewhere.

Under this assumption, the maximum image value directly reveals the illuminant:

$$\max_{x,y} I(x,y) = 1 \cdot L \implies \boxed{L = \max_{x,y} I(x,y)}.$$

Again, $\rho(x,y) = I(x,y)/L$.

Why Brightest=White Works (Sometimes)

This algorithm is even simpler than gray-world and is similarly defensible. Many natural scenes do contain a specular highlight, a piece of paper, a cloud, or a chrome reflection bright enough to act as a near-white reference.

But the algorithm is brittle. A single saturated pixel from a streetlight or an over-exposed highlight can completely set L , and any noise in the brightest part of the image is amplified into a global error. There is also no white surface visible at all in some scenes (a sunset, the inside of a cave), and yet we are still able to perceive color there.

3.3 Comparison on a Synthetic Example

A Uniform Image Tells the Two Algorithms Apart

Consider a uniform image

$$I = (100, 100, 100, 100).$$

Gray-world estimates

$$L = \frac{2}{4} (100 + 100 + 100 + 100) = 200, \quad \rho = \frac{100}{200} = 0.5.$$

The algorithm concludes that the surface is a uniform mid-gray, lit by a strong light.

Brightest=White estimates

$$L = \max(100, 100, 100, 100) = 100, \quad \rho = \frac{100}{100} = 1.0.$$

The algorithm concludes the surface is uniform white, lit by a weaker light.

Both answers are mathematically consistent with the image. They disagree about what shade of paint and what brightness of light were responsible for it.

What do humans see? Asked to look at a uniform patch in isolation, almost everyone reports it as *white*. This is the **Gelb effect**, and it tells us the brain is doing something much closer to brightest=white than to gray-world.

4 What Do Humans Use?

The Gelb effect is the historical name for the observation that an isolated patch in the visual field tends to look white – or close to white – regardless of its absolute luminance. Roughly: when there is no other reference, the brightest thing in view sets the top of the lightness scale.

Definition 4.1 The Gelb Effect

A patch shown in isolation – with no brighter surface visible nearby – is perceived as white or near-white, even when its physical luminance is low. The visual system anchors the top of its lightness scale to the brightest local reference rather than to any absolute luminance threshold.

This is suggestive evidence that humans use something *closer to* the brightest-pixel-is-white rule than to gray-world. But the truth is richer:

- Show a person an isolated Gelb patch, then introduce a genuinely white reference next to it, and the Gelb patch immediately darkens in perception. The brain re-anchors as soon as a brighter local reference becomes available.
- Humans appear to weight multiple cues at once – gray-world tendencies, brightest local patches, memory of familiar object colors, mutual inter-reflections between surfaces, specular highlights – rather than committing to any single rule.
- In a scene with strong contextual cues (a familiar object, a known light source, a piece of white paper in view), the brain’s color constancy is remarkable. In scenes where those cues are absent or contradictory, it can fail spectacularly, as we will see with the dress in Section 5.

Take-Home About Perception

Humans appear to use a variant of brightest-is-white, augmented with adaptive anchoring and far richer scene priors than either of the two textbook algorithms above. The textbook algorithms are caricatures of the brain’s strategy; the brain’s algorithm is the limit one approaches by stacking together many such heuristics, weighted by reliability cues.

5 Color Constancy: The RGB Extension

Everything so far has been scalar. The natural world has color, and the entire logic carries through channel by channel with essentially no extra ideas.

5.1 The Simplified Color Model

We give each surface a triple of albedos – one per color channel – and each illuminant a triple of intensities:

$$\begin{aligned}I_r(x, y) &= \rho_r(x, y) L_r \cos(\theta), \\I_g(x, y) &= \rho_g(x, y) L_g \cos(\theta), \\I_b(x, y) &= \rho_b(x, y) L_b \cos(\theta).\end{aligned}$$

Absorbing the geometric factor into the illuminant gives the clean per-channel product $I_c = \rho_c L_c$ for $c \in \{r, g, b\}$.

A surface is now characterized by its RGB albedo vector $\boldsymbol{\rho} = (\rho_r, \rho_g, \rho_b)$:

A Vocabulary of Albedos

A quick translation table between RGB albedo triples and the everyday color names we attach to them:

- $\boldsymbol{\rho} = (0.8, 0.8, 0.8)$ – white.
- $\boldsymbol{\rho} = (0.2, 0.2, 0.2)$ – black.
- $\boldsymbol{\rho} = (0.8, 0.2, 0.2)$ – red.
- $\boldsymbol{\rho} = (0.2, 0.8, 0.2)$ – green.
- $\boldsymbol{\rho} = (0.2, 0.2, 0.8)$ – blue.
- $\boldsymbol{\rho} = (0.8, 0.8, 0.2)$ – yellow (high red, high green, low blue).
- $\boldsymbol{\rho} = (0.2, 0.8, 0.8)$ – cyan (low red, high green, high blue).
- $\boldsymbol{\rho} = (0.8, 0.2, 0.8)$ – magenta.

This is exactly the additive color logic of an RGB display, run in reverse to describe the reflectance of a surface.

5.2 The Color Constancy Problem

Definition 5.1 Color Constancy

Given a color image $I_c(x, y)$ for $c \in \{r, g, b\}$ produced by the model

$$I_c(x, y) = \rho_c(x, y) \cdot L_c,$$

recover both the per-channel reflectance maps $\rho_c(x, y)$ and the illuminant color $\mathbf{L} = (L_r, L_g, L_b)$.

Counting again. We now have $3N$ observations (N pixels in each of 3 channels) and $3N + 3$ unknowns ($\boldsymbol{\rho}$ at each pixel plus a global illuminant triple). The shortfall is fixed at 3 unknowns – it does not grow with image size – but the ambiguity persists: scale up all albedos in a channel by a factor and divide L in that channel by the same factor and the image looks identical.

5.3 Algorithms: Just Run the Scalar Version Three Times

The simplest color-constancy algorithms apply gray-world or brightest=white *independently to each channel*.

Per-Channel Color Constancy

For each channel $c \in \{r, g, b\}$:

- **Gray-World:** $L_c = \frac{2}{N} \sum_{x,y} I_c(x, y).$
- **Brightest=White:** $L_c = \max_{x,y} I_c(x, y).$

Then recover $\rho_c(x, y) = I_c(x, y)/L_c.$

The implicit gray-world assumption is now “the average of the world is achromatic gray,” i.e., the per-channel means are equal. The implicit brightest=white assumption is “the brightest pixel in each channel comes from a common white surface.”

5.4 A Worked Example That Goes Wrong

Suppose the true illuminant is yellow:

$$\mathbf{L} = (180, 180, 90).$$

The scene contains two kinds of stripes:

- **Dark stripes:** $\rho = (0.33, 0.33, 0.33)$ – gray paint. Each channel produces

$$\mathbf{I}_{\text{dark}} = (0.33 \cdot 180, 0.33 \cdot 180, 0.33 \cdot 90) \approx (60, 60, 30).$$

- **Bright stripes:** $\rho = (0.5, 0.5, 1.0)$ – a cyan-ish paint with full blue reflectance. Each channel produces

$$\mathbf{I}_{\text{bright}} = (0.5 \cdot 180, 0.5 \cdot 180, 1.0 \cdot 90) = (90, 90, 90).$$

Now run brightest=white per channel on the observed image. The maximum in each channel is 90, so the algorithm reports

$$\hat{\mathbf{L}} = (90, 90, 90),$$

a perfectly *neutral* illuminant. Dividing through gives apparently sensible reflectances. But the truth was a yellow illuminant lighting a partly-cyan surface, and the algorithm has confidently re-assigned the yellow tint from the light to the surface. There is no way to tell from the image alone – the data are exactly the same as they would be for a different ground truth.

Why the Example Matters

The point of this example is not that brightest=white is a bad algorithm – it is that *the problem is genuinely ambiguous*. The same image is consistent with a yellow light on partly-cyan stripes, and with a white light on yellower stripes. Any algorithm must commit to one interpretation. The choice of prior *is* the choice of perceived color. This is the structural reason behind the next example.

5.5 The Famous Example: #thedress

In 2015 a low-quality photograph of a striped dress went viral when half the internet saw it as “black and blue” and the other half as “white and gold.” From the perspective of this lecture, the explanation is straightforward:

- Viewers who interpret the illuminant as *yellow* (warm indoor lighting, perhaps a tungsten bulb) divide the yellow tint out of the image and are left with “black and **purple** stripes” – popularly described as black and blue.
- Viewers who interpret the illuminant as *white* or even bluish (an outdoor shadow, say) leave the yellow tint in the surface and see “**gold and white** stripes.”

Why the Dress is Genuinely Interesting

The image is genuinely ambiguous. Both interpretations are mathematically consistent with the observed pixels. The pictorial cues that normally let the brain pin down the illuminant – a known-white reference somewhere in the frame, a familiar object whose color we already know, consistent context elsewhere in the image – are mostly absent or unreliable in that particular photograph. Different observers, looking at the same pixels, commit to different illuminant priors and report different surface colors.

This is the same ambiguity as the four-pixel image in Section 2, but in full color and shared on social media. The dress is not a curiosity; it is the white balance problem caught in the act of being ambiguous.

5.6 Less Famous but Equally Telling: The Yellowing Demonstration

A complementary demonstration works as follows. Take a photograph that contains a clear white reference – a piece of paper, say. Apply a uniform yellow tint to the whole image (multiply each pixel by a yellow vector). Now crop out the white reference and rescale the cropped region to fill the frame.

Two things happen to the observer.

- When the white reference is in view, the visual system anchors on it. The yellow cast is interpreted as a yellow *illuminant* and is mostly discounted – the surfaces look more or less their original colors.
- Once the white reference is cropped out, the anchor disappears. The brain commits to a different (whiter) illuminant prior and reads the residual yellow as belonging to the *surfaces*. The cropped image looks like it shows objects that are genuinely more yellow, not the original objects under yellow light.

This is the same phenomenon at work in the dress, with the manipulation under the experimenter’s control: removing the white anchor is enough to flip the perceptual decomposition.

6 Putting It All Together

6.1 Comparing Retinex and White Balance Side by Side

Two Different Slices of the Same Inverse Problem

- **Retinex** preserves the spatial structure of the illumination: $L = L(x, y)$. It uses a *spatial* prior – that reflectance edges are sharp and illumination edges are soft – to recover both maps from a single image.
- **White balance and color constancy** collapse the illumination to a single global value: L (or, in color, the triple $\mathbf{L}$). They use a *statistical* prior over the scene – the average is gray, or the brightest patch is white – to fix that global value.

Both attack the intrinsic-image programme of Barrow and Tenenbaum, but with different simplifications about what part of the inverse problem is hard. Retinex assumes lighting varies smoothly; white balance assumes it does not vary at all. Most real algorithms (and the brain itself) sit somewhere in between.

6.2 Beyond the Two Textbook Algorithms

Gray-world and brightest=white are pedagogically perfect because they are simple and explicit, but they are also caricatures. Real cameras and the visual system use much more sophisticated machinery, including:

- **Gamut mapping** – restricting illuminant estimates to the set of colors that could plausibly arise under known illuminants.
- **Bayesian color constancy** – explicit probabilistic priors over surface and illuminant colors, with the posterior $p(\boldsymbol{\rho}, \mathbf{L} \mid I)$ used to choose an estimate.
- **Specular highlight cues** – using the fact that a glossy specular reflection carries the color of the light source directly.
- **Familiar-object priors** – if the scene contains a banana or a face, the brain knows roughly what color they should be and can pin down the illuminant from there.
- **Learning-based methods** – modern computer vision systems train neural networks to estimate $\mathbf{L}$ end-to-end from raw images, learning the relevant priors from large datasets.

These are different ways of doing what gray-world and brightest=white are also trying to do: bring in an extra piece of information that breaks the $N + 1$ -versus- N ambiguity.

6.3 Summary

White Balance and Color Constancy in One Page

- **Model:** a single image is produced by $I(x, y) = \rho(x, y) \cdot L$ in grayscale, or $I_c = \rho_c L_c$ per channel in color, where ρ is surface albedo and L is a global illuminant.
- **Goal (white balance):** given I , recover both ρ and L .
- **Goal (color constancy):** the same problem run independently on the R, G, B

channels.

- **Fundamental ambiguity:** N observations versus $N + 1$ unknowns (or $3N$ vs $3N + 3$ in color). One extra assumption is mandatory – the problem cannot be solved without it.
- **Gray-world algorithm:** assumes mean reflectance is 0.5. Yields $L = \frac{2}{N} \sum I(x, y)$.
- **Brightest=white algorithm:** assumes the brightest pixel is a perfect white surface. Yields $L = \max I(x, y)$.
- **Human vision (Gelb effect):** appears closer to brightest=white than to gray-world, but uses much richer scene priors – familiar objects, specular highlights, contextual references.
- **The dress:** a real-world consequence – when the illuminant prior is ambiguous, different viewers commit to different illuminants and see different surface colors. The disagreement is not about pixel values; it is about which prior the brain happened to pick.